CSC3042F 2026 — Assignment 2

Encoder-Decoder Models for Grapheme-to-Phoneme Conversion

Student Number: PSWKAM001 | Date: May 2026 | Device: Kaggle

REPORT

AI Usage Statement

I used ChatGPT for debugging assistance and to clarify concepts, such as how to change directories, to provide code suggestions, and to create a table. All implementation decisions, written analysis, and conclusions are my own work. No section of the code or this report was generated **from scratch** by an AI tool.

1 Introduction

This report presents an implementation of encoder-decoder LSTM models for Grapheme-to-Phoneme (G2P) conversion. G2P conversion is a sequence-to-sequence modelling task, mapping a written word (a sequence of characters) to its pronunciation in 'ARPAbet' notation. A grid search over the learning rate, embedding dimension, and hidden size is used to identify the best hyperparameter configuration. Three model variants are then trained on the CMU Pronouncing Dictionary and compared: a **bottleneck baseline** with no context vector, a **fixed context vector** at every decoder step, and a **dynamic attention-based** context vector. The models are evaluated using Phoneme Error Rate (PER) and word accuracy.

2 OOP Design and Implementation

1.1 Architecture Overview

The implementation follows a clean object-oriented hierarchy. Every model component is an nn.Module subclass, and training/evaluation logic is separated into pure functions. No nn.LSTM, nn.LSTMCell, or other built-in PyTorch RNN modules are used anywhere.

Class	Responsibility
`LSTMCell`	The LSTMCell class processes a single timestep of sequential data, producing an updated hidden state and cell state. It implements the six LSTM gating equations from scratch, using nn.Linear layers for the weight matrices W and U, and nn.Parameter for the bias vectors b.
`ContextLSTMCell`	The ContextLSTMCell, extends the base cell with additional V weight matrices (one per gate) that project the context vector z into the hidden space. This is used in Setups 2 and 3. Keeping it as a separate class avoids cluttering LSTMCell with conditional logic while allowing the two cell types to be composed cleanly inside the Decoder.
`Encoder`	The Encoder takes a sequence of character indices as input, converts them via an embedding layer into dense vectors, and processes them through stacked LSTMCell layers. The embedding layer maps each character index to a learned dense representation. The encoder returns all hidden states across every timestep, as well as the final hidden and cell states
`Decoder`	The Decoder class takes the encoder's final hidden and cell states as its initial state, along with the target phoneme sequence and optionally the encoder outputs, and generates the predicted phoneme sequence one token at a time. A key design decision was implementing all three setups within a single Decoder class, controlled by a context mode parameter, rather than three separate classes

Seq2Seq	The Seq2Seq class wraps the Encoder and Decoder into a single model. During training, it uses teacher forcing, where the ground-truth target phoneme tokens are fed as decoder input at each step, stabilising training by preventing error accumulation. During inference, greedy decoding is used; one phoneme is generated at a time by taking the argmax of the output logits, feeding each predicted token back as the next input, and stopping when '<EOS>' is predicted or the maximum length is reached.

3 Hyperparameter Tuning

Setup 1 (bottleneck baseline) was trained for 9 epochs per configuration. The grid searched three values each for learning rate, embedding dimension, and hidden size (27 configurations total). Validation PER was the selection criterion.

LR	Embed dim	Hidden size	Val PER	Val Word Acc
5e-4	**128**	**256**	**Best ✓**	**Highest ✓**
1e-3	128	256	2nd	2nd
1e-4	64	128	3rd	3rd
1e-4	32	64	Worst	Worst

Selected configuration: lr = 5e-4, embed_dim = 128, hidden_size = 256. Larger hidden sizes provided more capacity to encode the irregular spelling patterns in English. The learning rate of 5e-4 struck the best balance, 1e-3 caused occasional instability in the from-scratch LSTM, while 1e-4 converged too slowly within the 9-epoch budget. This configuration was used for all three setups in the final comparison.

4 Architecture Comparison and Results

4.1 Training Setup

All three setups were trained under identical conditions: lr = 5e-4, embed_dim = 128, hidden_size = 256, batch_size = 128, seed = 42, max_epochs = 30, early stopping patience = 5 (val loss). Cross-entropy loss with ignore_index = 0 (PAD) was used. Adam optimiser with gradient clipping (max_norm = 1.0) at every step.

4.2 Loss Curves

The line graphs in the notebook show training and validation loss for all three setups on shared axes. Setup 3 (attention) achieves the lowest final validation loss and converges most smoothly. Setup 2 (fixed context) is a clear improvement over Setup 1 (bottleneck). Setup 1 shows slightly more instability in its loss curve, consistent with the hidden-state bottleneck forcing all source information through a single vector.

4.3 Test Set Evaluation

Setup	Test PER	Word Accuracy (%)
Setup 1 - No context (bottleneck baseline)	0.1162	62.01

Setup 2 - Fixed context vector	0.0843	70.24
Setup 3 - Dot-product attention	**0.0671**	**74.89**

4.4 Discussion

Convergence speed. Setup 3 (attention) reached its best validation loss fastest; it benefits from being able to re-examine the encoder outputs at every decode step, so it never has to rely on a single compressed vector. Setup 1 is slowest to converge and plateaus at higher loss.

Setup 2 vs Setup 1. The fixed context vector reduces PER from 0.1162 to 0.0843; a relative improvement of 27.5% and word accuracy improves from 62.0% to 70.2%. Simply re-injecting the final encoder hidden state at every decode step meaningfully helps: the decoder no longer has to remember all source information through its own recurrence alone.

Setup 3 vs Setup 2. Attention reduces PER further from 0.0843 to 0.0671 and pushes word accuracy to 74.9%. The dynamic context vector allows the decoder to focus on the specific characters most relevant to each output phoneme, rather than receiving the same global summary z at every step. This is particularly impactful for longer words where the fixed context must compress more information.

Error analysis. Words where Setup 1 fails but Setup 3 succeeds tend to be longer multi-syllable words (e.g. 'psychological', 'straightforward', 'ethelinda') and words with highly irregular spelling–sound correspondences (e.g. words containing 'ough', 'tion', 'eau'). In these cases, the bottleneck's single compressed vector is insufficient, and Setup 3's attention can focus on the relevant characters for each phoneme independently.